GenSpark

Lists and Nested Lists (needed for the Tic-Tac-Toe board)

Lists and Nested Lists

So far your variables have held one value each: a number, a piece of text, or a `True` or `False`. Real programs work with collections of values: a board with nine squares, a list of menu choices, every record in a file. The list is Python's most common way to hold a collection. This article teaches you to create, read, change, and loop over lists, then to nest lists inside lists to build a grid. That grid is the Tic-Tac-Toe board. Dictionaries and sets come later, in the Collections and Data Modeling article in Week 3.

What a List Is

A list is an ordered collection of values, written in square brackets with commas between the items:

python

```
numbers = [10, 20, 30, 40, 50]
names = ["Alex", "Sam", "Priya"]
mixed = [1, "two", 3.0, True]      # a list can mix types, though it usual
empty = []                         # an empty list
```

Three properties matter:

- **Ordered.** The items stay in the order you put them in.
- **Indexed.** Each item has a position number you can use to reach it.
- **Changeable.** You can add, remove, and replace items after the list exists. The technical word is *mutable*.

You have already used lists in a few places, such as `choice in ["1", "2", "3"]` and `random.choice(["red", "green", "blue"])`.

Indexing

Each item has an **index**, its position in the list. Counting starts at 0, not 1:

python

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])      # 10, the first item
print(numbers[1])      # 20
print(numbers[4])      # 50, the last item
```

Negative indexes count from the end. `-1` is the last item:

python

```
print(numbers[-1])     # 50
print(numbers[-2])     # 40
```

Use `len()` to find how many items a list holds. The valid indexes run from `0` to `len(numbers) - 1`:

python

```
print(len(numbers))    # 5
```

Reaching past the end is an error:

python

```
print(numbers[5])
```

```
IndexError: list index out of range
```

You will see `IndexError` often, and it is nearly always an off by one mistake like the ones you met with `range()`. The same error appears if you ask for the first item of an empty list.

Slicing

A **slice** pulls out part of a list. Write `[start:stop]`. It includes `start` and stops **before** `stop`, the same rule as `range()`:

python

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:3])     # [20, 30]
print(numbers[:2])      # [10, 20], from the start
print(numbers[3:])      # [40, 50], to the end
print(numbers[::2])     # [10, 30, 50], every second item
print(numbers[::-1])    # [50, 40, 30, 20, 10], reversed
```

A slice always makes a **new list**. It never raises an `IndexError`, even when the numbers go past the end.

Changing a List

Because lists are mutable, you can replace an item by assigning to its index:

python

```
numbers[0] = 99
print(numbers)      # [99, 20, 30, 40, 50]
```

These methods add and remove items:

Method	What it does	Example
`append(x)`	Adds x to the end	`items.append(4)`
`insert(i, x)`	Adds x at position i	`items.insert(0, 4)`
`extend(other)`	Adds every item from another list	`items.extend([5, 6])`
`remove(x)`	Removes the first item equal to x	`items.remove(3)`
`pop()`	Removes and returns the last item	`last = items.pop()`
`pop(i)`	Removes and returns the item at i	`first = items.pop(0)`

python

```
items = [1, 2, 3]
items.append(4)          # [1, 2, 3, 4]
items.insert(0, 0)       # [0, 1, 2, 3, 4]
items.extend([5, 6])     # [0, 1, 2, 3, 4, 5, 6]
items.remove(3)          # [0, 1, 2, 4, 5, 6]
last = items.pop()       # last is 6, items is [0, 1, 2, 4, 5]
```

`append`, `insert`, `extend`, and `remove` change the list itself and return nothing useful (they return `None`). `pop` changes the list too, and also hands back the item it removed. Some of these raise errors when they can't do their job:

```
ValueError: list.remove(x): x not in list
IndexError: pop from empty list
```

Check with `in` or `len()` first if the item might not be there.

Handy Tools for Lists

python

```
numbers = [4, 2, 8, 2]

print(len(numbers))        # 4
print(2 in numbers)        # True
print(numbers.count(2))    # 2, how many times 2 appears
print(numbers.index(8))    # 2, the position of the first 8
print(min(numbers))        # 2
print(max(numbers))        # 8
print(sum(numbers))        # 16
```

`numbers.index(x)` raises `ValueError: x is not in list` when the item is missing, so test with `in` first.

Sorting: sort() and sorted()

python

```
scores = [3, 1, 2]

print(sorted(scores))      # [1, 2, 3], a NEW sorted list
print(scores)              # [3, 1, 2], unchanged

scores.sort()              # sorts the list itself
print(scores)              # [1, 2, 3]
```

A trap catches almost every beginner. `sort()` changes the list and returns `None`:

python

```
scores = [3, 1, 2]
scores = scores.sort()     # WRONG: scores is now None
```

Use `sorted(scores)` when you want a new list, and plain `scores.sort()` (not assigned to anything) when you want to change it in place.

An empty list counts as False

An empty list is treated as `False` in a condition, and a list with any items is treated as `True`:

python

```
moves_left = []

if not moves_left:
    print("No moves left")     # this runs
```

This is a clean way to ask "is anything left?", and you will use it to detect a full Tic-Tac-Toe board.

Lists and text

`split()` turns a string into a list, and `join()` turns a list of strings back into one string:

python

```
words = "red,green,blue".split(",")     # ['red', 'green', 'blue']
print(" - ".join(words))                # red - green - blue
print(list("abc"))                      # ['a', 'b', 'c']
```

`join()` only works when every item is a string. A list of numbers raises `TypeError: sequence item 0: expected str instance, int found`. Convert with `str()` first.

Looping Over a List

A `for` loop visits each item in turn. You already know this from the loops article:

python

```
names = ["Alex", "Sam", "Priya"]

for name in names:
    print(f"Hello, {name}!")
```

When you need the position as well as the item, use `enumerate()`:

python

```python
for position, name in enumerate(names):
    print(position, name)       # 0 Alex, 1 Sam, 2 Priya

for number, name in enumerate(names, start=1):
    print(number, name)         # 1 Alex, 2 Sam, 3 Priya
```

The second form with `start=1` is useful for menus that show numbers starting from 1.

If you really need the index to change items, loop over `range(len(names))`:

python

```python
for i in range(len(names)):
    names[i] = names[i].upper()
```

Do not change a list while you loop over it

Removing items from the list you are looping over makes Python skip items:

python

```python
numbers = [2, 4, 6, 8]

for n in numbers:
    if n % 2 == 0:
        numbers.remove(n)

print(numbers)      # [4, 8], not []
```

When an item is removed, the ones after it shift down, and the loop moves on without checking the item that took its place. Build a new list instead:

python

```python
numbers = [2, 4, 6, 8]
```

```
odds = [n for n in numbers if n % 2 != 0]
```

That second line is a list comprehension.

List Comprehensions

A **list comprehension** builds a new list in one line. It reads like a sentence: "give me *this*, for each *item* in *that*, if *condition*."

python

```
doubled = [n * 2 for n in range(5)]                # [0, 2, 4, 6, 8]
evens = [n for n in range(10) if n % 2 == 0]       # [0, 2, 4, 6, 8]
shout = [word.upper() for word in ["hi", "yo"]]    # ['HI', 'YO']
```

The same thing written as a loop:

python

```
doubled = []
for n in range(5):
    doubled.append(n * 2)
```

Both are correct. Use a comprehension when it stays short and easy to read. If it gets long or complicated, write the loop.

any() and all()

Two built-in functions answer yes or no questions about a whole collection:

- `all(...)` is `True` when **every** item is true.
- `any(...)` is `True` when **at least one** item is true.

python

```
print(all(n > 5 for n in [6, 7, 9]))     # True
```

```
print(all(n > 5 for n in [1, 9]))        # False
print(any(n > 5 for n in [1, 9]))        # True
print(any(n > 5 for n in [1, 2]))        # False
```

The part inside the parentheses, `n > 5 for n in [1, 9]`, works like a comprehension without the square brackets. Together they ask a question about every item. Two edge cases are worth remembering: `all()` of an empty collection is `True`, and `any()` of an empty collection is `False`.

`all()` is perfect for checking a winning line in Tic-Tac-Toe: "is every cell in this row an X?"

Tuples

A **tuple** is like a list that cannot be changed. It is written with parentheses:

python

```
move = (1, 2)
row, col = move          # unpacking: row is 1, col is 2
print(move[0])           # 1
```

Trying to change one fails:

python

```
move[0] = 5
```

```
TypeError: 'tuple' object does not support item assignment
```

Use a tuple for a small fixed group that belongs together, such as a `(row, col)` position on a board. A list of positions is then `[(0, 0), (0, 1), (2, 2)]`. Strings are also unchangeable. `word[0] = "b"` raises the same kind of `TypeError`, and you make changes by building a new string.

Copies and Aliases

This is the part of lists that surprises people most. Assigning a list to another variable does **not** copy it. Both names point to the same list:

python

```
a = [1, 2, 3]
b = a              # b is another name for the SAME list
b.append(4)

print(a)           # [1, 2, 3, 4]
print(b)           # [1, 2, 3, 4]
print(a is b)      # True
```

Think of a list as a document and a variable as a label on it. `b = a` sticks a second label on the same document. If you want an independent copy, make one:

python

```
a = [1, 2, 3]
c = a.copy()       # a new list with the same items
c.append(5)

print(a)           # [1, 2, 3]
print(c)           # [1, 2, 3, 5]
```

`a[:]` makes a copy too, because a full slice builds a new list.

The same idea applies when you pass a list to a function. The function receives the **same list**, so if it changes it, the caller sees the change:

python

```
def add_item(items, new_item):
    items.append(new_item)       # changes the caller's list

shopping = ["milk"]
add_item(shopping, "eggs")
print(shopping)                  # ['milk', 'eggs']
```

That is sometimes what you want, and sometimes a nasty surprise. When a function changes a list it was given, say so in its docstring.

Nested Lists

A list can hold other lists. A list of lists is a table, or a grid:

python

```
board = [
    ["X", " ", "O"],
    [" ", "X", " "],
    ["O", " ", " "],
]
```

Each inner list is a **row**. To reach one cell, use two indexes: the row first, then the column:

python

```
print(board[0])        # ['X', ' ', 'O'], the whole first row
print(board[0][2])     # 'O', row 0, column 2
print(board[1][1])     # 'X', the center

board[2][2] = "X"      # change the bottom right cell
```

Read `board[row][column]` from left to right: first pick the row, then pick the cell inside it. Both numbers start at 0. A 3 by 3 board has rows and columns numbered 0, 1, and 2.

Building a grid, and a trap

To make a board of any size, you might try this:

python

```
board = [[" "] * 3] * 3       # WRONG
board[0][0] = "X"
print(board)
```

```
[['X', ' ', ' '], ['X', ' ', ' '], ['X', ' ', ' ']]
```

All three rows changed. The outer `* 3` made three references to the **same** inner list, so there is really only one row, shown three times. This is the aliasing problem again, and it is a very common bug in grid programs.

The fix is a comprehension that builds a new row each time:

python

```
board = [[" "] * 3 for _ in range(3)]       # RIGHT
board[0][0] = "X"
print(board)
```

```
[['X', ' ', ' '], [' ', ' ', ' '], [' ', ' ', ' ']]
```

Here `[" "] * 3` is fine, because it builds one row of three separate string values. The problem was only multiplying the outer list. The `_` is a variable name that means "I don't need this value". Wrap it in a function so any size works:

python

```
def new_board(size):
    """Return a size by size board filled with empty cells."""
    return [[" "] * size for _ in range(size)]
```

Looping through a grid

Use a loop inside a loop, as you saw in the loops article. The outer loop picks a row, and the inner loop visits each column:

python

```
size = len(board)
```

```
for row in range(size):
    for col in range(size):
        print(row, col, board[row][col])
```

Or loop over the rows directly when you do not need the numbers:

python

```
for row in board:
    for cell in row:
        print(cell)
```

Reading columns and diagonals

Rows are easy because each one is a list. A column takes a comprehension that picks the same position from every row:

python

```
column_0 = [board[row][0] for row in range(len(board))]
```

The two diagonals follow a pattern. On a board of `size` rows:

python

```
main_diagonal = [board[i][i] for i in range(size)]                 # top l
other_diagonal = [board[i][size - 1 - i] for i in range(size)]     # top r
```

Copying a grid

Copying a nested list needs extra care. `board.copy()` copies only the outer list, so the rows are still shared:

python

```
copy_of_board = board.copy()        # shallow copy: rows are shared
copy_of_board[2][2] = "Z"
print(board[2][2])                  # 'Z', the original changed too!
```

Copy each row to get a real copy:

python

```
independent = [row[:] for row in board]
```

For deeper nesting, Python's `copy` module has `copy.deepcopy(board)`, which copies every level. A board with two levels only needs the one line above.

The Hard AI in Tic-Tac-Toe needs to try moves without ruining the real board. There are two ways to do it: work on an independent copy, or place a mark, evaluate it, and then put the empty cell back (an "undo"). Both rely on knowing exactly which parts of your data are shared.

Building the Pieces of a Board

Here are the building blocks for a Tic-Tac-Toe board. Each function is small, does one job, and takes the board as a parameter, as the functions article recommended.

python

```
EMPTY = " "


def new_board(size):
    """Return a size by size board filled with empty cells."""
    return [[EMPTY] * size for _ in range(size)]


def place_mark(board, row, col, mark):
    """Put mark at row, col. Return False if the cell is taken."""
    if board[row][col] != EMPTY:
        return False
    board[row][col] = mark
    return True
```

```
def empty_cells(board):
    """Return a list of (row, col) pairs for every empty cell."""
    size = len(board)
    return [(r, c) for r in range(size) for c in range(size) if board[r][c
```

```
def has_full_line(board, mark):
    """Return True if mark fills a whole row, column, or diagonal."""
    size = len(board)
    for row in board:
        if all(cell == mark for cell in row):
            return True
    for col in range(size):
        if all(board[row][col] == mark for row in range(size)):
            return True
    if all(board[i][i] == mark for i in range(size)):
        return True
    return all(board[i][size - 1 - i] == mark for i in range(size))
```

```
def print_board(board):
    """Print the board with lines between the cells."""
    separator = "-+-".join("-" * len(board))
    for i, row in enumerate(board):
        print(" | ".join(row))
        if i < len(board) - 1:
            print(separator)
```

Try them together:

python

```
board = new_board(3)
place_mark(board, 0, 0, "X")
place_mark(board, 1, 1, "O")
place_mark(board, 0, 1, "X")
print(place_mark(board, 0, 1, "O"))     # False, the cell is taken
place_mark(board, 0, 2, "X")
```

```
print_board(board)
print("Empty cells:", len(empty_cells(board)))
print("X wins:", has_full_line(board, "X"))
print("O wins:", has_full_line(board, "O"))
```

Output:

```
False
X | X | X
--+---+--
  | O |
--+---+--
  |   |
Empty cells: 5
X wins: True
O wins: False
```

A few things to notice:

- `place_mark` changes the board that was passed in, and returns `True` or `False` to say whether it worked. A caller can use that to reject a move without crashing.
- `empty_cells` returns a list of tuples. If the list is empty, the board is full, and `if not empty_cells(board):` detects a draw.
- `has_full_line` checks a *complete* row, column, or diagonal. That is the winning rule on a 3 by 3 board and on a 4 by 4 board where four in a row wins. When a win needs fewer marks than the board is wide, as on the 5 by 5 board in the lab, you must check every stretch of marks along each line instead. The same tools apply: slices and `all()`.
- `random.choice(empty_cells(board))` picks a random empty cell for the Easy AI. If the list is empty, it fails with `IndexError: Cannot choose from an empty sequence`. Check for a full board first.

Rows and columns start at 0

Players think in rows and columns numbered from 1, but Python counts from 0. When the player types `2 3`, the code must use `board[1][2]`. Subtract 1 when you read the input, and add 1

when you display a position. Keeping that conversion in one place prevents most off by one mistakes in grid games.

Common Mistakes

Mistake	What happens	Fix
Using index 1 for the first item	Everything is off by one	The first index is `0`
Asking for `items[len(items)]`	`IndexError`	The last index is `len(items) - 1`, or use `-1`
`b = a` to copy a list	Both names change together	Use `a.copy()` or `a[:]`
`scores = scores.sort()`	`scores` becomes `None`	Call `scores.sort()` alone, or use `sorted()`
Removing items while looping over the list	Items are skipped	Build a new list with a comprehension
`[[" "] * 3] * 3` for a grid	All rows change together	`[[" "] * 3 for _ in range(3)]`
`board.copy()` on a nested list	The rows are still shared	`[row[:] for row in board]`
Mixing up `board[row][col]` order	You read the wrong cell	Row first, then column
Calling `remove` or `pop` on a missing item or empty list	`ValueError` or `IndexError`	Check with `in` or `len()` first
`"-".join([1, 2])`	`TypeError`	Convert items with `str()` first
`random.choice([])`	`IndexError`	Check that the list is not empty
A function changes a list and the caller does not expect it	Surprise changes	Document it, or work on a copy

Try It Yourself

Create a file called `lists_practice.py` and run it with `python3 lists_practice.py`. Try these:

1. Make a list of five numbers. Print the first, the last, and the middle item. Then print `numbers[5]` and read the `IndexError`.
2. Print slices of your list: the first three items, the last two, every other item, and the whole list reversed.
3. Start with an empty list. Use `append` to add three names, `insert` to put a name at the front, and `pop` to remove the last one. Print the list after each step.
4. Write a loop that prints each name in your list with a number starting from 1, using `enumerate`.
5. Write a program that asks for five numbers, stores them as integers in a list, and prints the total, the smallest, the largest, and the average.
6. Make a list `[3, 1, 2]`. Sort it with `sorted()` and then with `.sort()`. Then write `result = numbers.sort()` and print `result`.
7. Write a list comprehension that builds the squares of 1 to 10. Then one that keeps only the squares that are even.
8. Create `b = a` for a list, change `b`, and print both. Then repeat with `b = a.copy()`.
9. Write a function `add_item(items, item)` that appends to a list. Call it and show that the original list changed.
10. Build a 3 by 3 board the wrong way (`[[" "] * 3] * 3`), change one cell, and describe what happens. Then build it the right way.
11. Write functions `get_row(board, r)`, `get_column(board, c)`, and `get_diagonals(board)` that return lists of cells.
12. Type in the board building blocks from this article, play a short game by calling `place_mark` for X and O, and print the board after each move.
13. Add a function `is_full(board)` that uses `empty_cells`, and a function `is_draw(board)` that is true when the board is full and neither player has a full line.

14. Make a copy of a board with `board.copy()`, change the copy, and show that the original changed. Then fix it with `[row[:] for row in board]`.

15. Extend `has_full_line` so it also reports *which* line won: return a string such as `"row 0"` or `"diagonal"`, or `None` if nobody has won.

Why This Matters for an FDE

Almost every real data task starts with a collection: rows from a database, records in a file, items in an API response, results from a search. Knowing how to select pieces, filter, and transform a list is the everyday core of Python work. The aliasing problems in this article, where two names secretly share one list, cause real bugs in production systems, and they are hard to spot unless you know to look for them. Treating a grid as a list of lists is also your first step toward working with tables and matrices of data.

Comprehension Check

1. What are the first and last valid indexes of a list with 6 items? What does `items[-1]` return?

2. What does `numbers[1:4]` return for `[10, 20, 30, 40, 50]`? What does `numbers[::-1]` return?

3. What does `scores.sort()` return, and what goes wrong with `scores = scores.sort()`?

4. After `b = a` and `b.append(9)`, what does `a` contain? Why?

5. Why does `board = [[" "] * 3] * 3` make all three rows change at once, and what is the fix?

6. In `board[1][2]`, what does each number mean? Which cell is `board[0][0]`?

7. Why is `board.copy()` not enough for a list of lists, and what do you write instead?

8. What does `all(cell == "X" for cell in row)` check? What does it return for an empty row?

9. A player types the row and column `3 1`. What indexes do you use to reach that cell in `board`?

10. Why is removing items from a list while looping over it risky, and what is a safer way?

← Previous: Functions & Modular Design

Next: OOP principles →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!